Stolpe,
Graal-LSP

Graal-LSP: A Zero-overhead Polyglot Language Server Implementation for the GraalVM

Graal-LSP: Eine Overhead-freie Language Server Implementierung für die GraalVM

by

Daniel Stolpe

A thesis submitted to the
Hasso Plattner Institute
at the University of Potsdam, Germany
in partial fulfillment of the requirements for the degree of

Master of Science in IT-Systems Engineering

Supervisors

Prof. Dr. Robert Hirschfeld
Dr. John Doe

Software Architecture Group
Hasso Plattner Institute
University of Potsdam, Germany

October 18, 2018

Abstract

The english abstract.

Zusammenfassung

Die Zusammenfassung auf deutsch.

Acknowledgments

I owe everything to my cat.

Contents

1. Introduction **1**
1.1. Contributions . . . 1
1.2. Structur of this Thesis . . . 2

2. Context **3**
2.1. Terms and Concepts . . . 3
2.2. Truffle and GraalVM . . . 3
2.3. Language Server Protocol . . . 3

3. Approach **5**
3.1. A Language-agnostic Language Server . . . 5
3.1.1. Concepts to support core LSP features . . . 5
3.2. Use of existing Truffle-APIs . . . 6
3.3. What is missing? . . . 6
3.3.1. Static information . . . 6
3.3.2. Proposed Truffle API changes . . . 7
3.4. How to deal with dynamically typed languages? -> Using runtime information . . . 7
3.5. Mapping LSP language server API to Truffle API . . . 7
3.6. Architecture . . . 7

4. Implementation **9**
4.1. Language Server Implementation . . . 9
4.1.1. Architecture . . . 9
4.1.2. Using runtime information . . . 9
4.1.3. LSP features implementation overview . . . 9
4.1.4. LSP client - VSCode extension example . . . 11
4.2. Requirements on Truffle Language Implementations . . . 11

5. Experience report **13**
5.1. Between IDE and Language Workbench . . . 13
5.2. Completion response time . . . 13
5.3. Limitations . . . 13
5.3.1. VSCode: Need to define supported languages in extension . . . 13
5.3.2. VSCode: Syntax highlighting defined at client-side . . . 13
5.3.3. LSP: No incremental sending of (completion) results . . . 13
5.3.4. Server: Only TCP (connecting to already running server), no std-in/-out impl . . . 13

5.3.5. Truffle: Only first syntax error is reported. Lang parsers fail on first error 13

6. Related Work **15**
6.1. Graal 15
6.2. Squimera 15
6.3. Language Workbenches 15
6.4. Run-time type collection 15

7. Conclusion and Future Work **17**

A. First Unimportant stuff. **21**

List of Figures

List of Tables

List of Listings

List of Abbreviations

LSP Language Server Protocol
IDE Integrated Development Environment

1. Introduction

- language development is challenging task
- what makes a good language: amongst performance, std lib, ecosystem and community, tools (for language developers and users) are important
- who provides the tools? language developers vs. language users
- normally tools are developed for a specific language and cannot be reused or need lots of adaptation
- In the Truffle framework, tooling (instrumentation) is part of the framework and therefore tools are implemented language-agnostic once
- Truffle has already some language-agnostic tools implemented (debugger, profiler, ...) and new ones can be easiliy built
- Truffle tooling support does not interfere with implementing high-performance languages
- Compared to a full IDE, some tools are missing
- Implementing LSP can provide further IDE-like features
- We propose an approach to build a generic LSP server on top of Truffle with:
  - minimal to zero language runtime performance overhead
  - minimal API changes to Truffle
  - minimal impl and maintainance overhead for language developers
- Benefits for the language developers: get IDE-like tools with minimal effort
- Benefits for language users: can choose from wide range of LSP clients + consistent dev-tools experience for different Truffle languages
- As proof of concept, we have implemented Graal-LSP- A Zero-overhead Polyglot Language Server Implementation for the GraalVM
- ?: focus on dynamically type languages? Or is this focus already included in Truffle (most impls are dynamically type langs)?

1.1. Contributions

- Contribution in detail:
  - An approach to build a generic LSP server on top of Truffle
  - An implementation of said approach: Graal-LSP
  - Proposals to extend the Truffle Framework with new interfaces needed to support more LSP features
  - An experience report how well existing Graal/Truffle-implemented languages work with Graal-LSP(with and without implementing the said interface proposals)

1.2. Structur of this Thesis

2. Context

2.1. Terms and Concepts

- stuff needed to understand Truffle instrumentation + Approach (using SL or Python as example):
  - AST (+mapping to source section)
  - AST Interpreter
  - frames
  - scopes

2.2. Truffle and GraalVM

- Truffle language implementation framework
  - Truffle instrumentation
  - Loading languages (dynamic classpath detection)
- GraalVM - high-performance polyglot VM

2.3. Language Server Protocol

- definition
- URIs to identify a source file and Locations
- client + server
- JSON-RPC
- graphic of exemplary LSP communication
- lsp4j - Java binding for LSP
  - version
  - map to Java types
  - JSON-RPC wrap
- LSP client: VSCode example

3. Approach

- language implementaion + tool development = challenging task
- we propose: implement LSP for Truffle
- typically one specific language server impl for one language
- Our approach: a language-agnostic language server impl for languages created by the language implementation framework Truffle

3.1. A Language-agnostic Language Server

- LSP enables arbitrary language clients to communicate with arbitrary language servers
- Normally a language server provides support for one specific language only
- To build a language-agnostic language server for Truffle we have to define what Truffle languages need to provide in general to support LSP
- LSP is trade-off - there are lang-specific IDEs with more/other features
- Therefore our solution will also be a trade-off using general language features available for all Truffle languages

3.1.1. Concepts to support core LSP features

LSP Features (version 3.7.0) we want to support:

- completion
- hover
- signatureHelp
- definition
- references
- documentHighlight
- documentSymbol
- symbol

Completion

- What does a Truffle language need to provide?
- mapping of caret (line+column) to an AST node of the language
- therefore code needs to be parsable
- different completion kinds: locals+globals vs object property completion
- via checking completion characters -> language needs to tell us its completion trigger chars
- globals/locals

3. *Approach*

  - language needs to derive a scope from an AST node
  - if not parsable -> only global scope possible
  - list valid variables in the scope + kind if known

- object property completion

  - code needs to be parsable
  - assumption: insterting a completion character => unparsable code
  - fix code by undoing last change + parse again
  - derive type information from node at caret
  - easy for statically type languages - not for dynamic

  Not implemented:

- completion resolve
- typeDefinition
- implementation
- codeAction
- documentLink
- documentLink resolve
- documentColor
- colorPresentation
- formatting
- rangeFormatting
- onTypeFormatting
- rename
- foldingRange

3.2. Use of existing Truffle-APIs

- what is already there?
- parsing sources
- AST
- scopes (global/local)
- frames and frame descriptors
- instrumentation
- -> functionality to support impl of debugger

3.3. What is missing?

3.3.1. Static information

- AST design focused on runtime, lacking many parse-time infos

  - mark declarations
  - mark variable read/write locations
  - ...

  - in general: semanitc relations between nodes
- most lang impls: no syntax-error tolerant parser

3.3.2. Proposed Truffle API changes

- language API
- Tags

3.4. How to deal with dynamically typed languages? -> Using runtime information

- Observe execution via instrumentation
- Collecting coverage data
- Side-effects -> Truffle-sandboxing
- Evaluation strategies:
  - Literals
  - Coverage data
  - Run-to-section
  - Globals

3.5. Mapping LSP language server API to Truffle API

- diagnostics: parsing: throws exception on syntax error -> map to to source range -> send diagnostic, fail-on-first-error

3.6. Architecture

- graphic showing launcher, lsp server, truffle api communication
- comparison to other language server impls (built on top of language)

4. Implementation

4.1. Language Server Implementation

- lsp4j - Java binding for LSP (see Context)

4.1.1. Architecture

- Truffle-API + Graal-SDK
- Launcher, Instrument, Server
- lsp4j adapters

4.1.2. Using runtime information

Implementation of:

- Observe execution via instrumentation
- Side-effects -> Truffle-sandboxing
- Nested Contexts
- Evaluation strategies:
  - Literals
  - Coverage data
    - storing frames (copies)
    - source location format
    - migrating coverage data on source edit
  - Run-to-section
  - Globals

4.1.3. LSP features implementation overview

Feature impl matrix:

- TextDocumentService interface
  - completion
    - locals+globals vs object property completion (via checking completion character)
    - locals+globals completion
      - find nearest node

4. *Implementation*

    - check coverage data
    - globals only fallback
    - Used Truffle-API: findLocalScopes() + findGlobalScopes() + (parseInline() for coverage) + ForeignAccess.sendIsExecutable() + ForeignAccess.sendIsInstantiable() + ForeignAccess.sendIsNull()
    - Used new Truffle-API: env.boxPrimitive() + GetSignature + GetDocumentation

  * object property completion

    - fixing source code (undo last edit + re-parse)
    - find nearest node
    - tryDifferentEvalStrategies() (literals -> coverage data -> run-to-section -> globals)
    - Used Truffle-API: findLocalScopes() + findGlobalScopes()

  – hover
  – signatureHelp
  – definition
  – typeDefinition
  – implementation
  – references
  – documentHighlight
  – documentSymbol
  – codeAction
  – codeLense
  – formatting
  – rangeFormatting
  – onTypeFormatting
  – rename
  – didOpen
  – didChange
  – didClose
  – didSave
  – willSave
  – willSaveWaitUntil
  – documentLink
  – documentLinkResolve
  – documentColor
  – colorPresentation

- WorkspaceService interface

  – executeCommand
  – symbol
  – didChangeConfiguration
  – didChangeWatchedFiles

- didChangeWorkspaceFolders

4.1.4. LSP client - VSCode extension example

4.2. Requirements on Truffle Language Implementations

- Implement existing API:
  - ExpressionTag + StatementTag
  - findTopScopes()
  - findLocalScopes()
  - findMetaObject()
  - findSourceLocation()
  - parseInline()
- Implement proposed/new API:
  - getCompletionTriggerCharacters()
  - getSignatureHelpCharacters() (TODO)
  - boxPrimitive()
  - GetSignature + GetDocumentation custom message
  - DefinitionTag + ReadVariableTag + WriteVariableTag

5. Experience report

5.1. Between IDE and Language Workbench

Language Workbench focus/aim: Efficient language (DSL) development, performance is not main focus vs. Graal/Truffle aim: Efficient execution

Assumption: Language Workbenches use static information only, our Truffle-based solution uses dynamic type information

5.2. Completion response time

5.3. Limitations

own chapter? or in cha:implementation?

5.3.1. VSCode: Need to define supported languages in extension

5.3.2. VSCode: Syntax highlighting defined at client-side

5.3.3. LSP: No incremental sending of (completion) results

5.3.4. Server: Only TCP (connecting to already running server), no std-in/-out impl

5.3.5. Truffle: Only first syntax error is reported. Lang parsers fail on first error

6. Related Work

6.1. Graal

[5]

6.2. Squimera

[4] [3]

6.3. Language Workbenches

[1]

6.4. Run-time type collection

[2]
CTags, SemanticDB

7. Conclusion and Future Work

Bibliography

[1] Sebastian Erdweg, Tijs Van Der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, et al. "The State of the Art in Language Workbenches". In: *International Conference on Software Language Engineering*. Springer. Springer International Publishing, 2013, pp. 197–217. DOI: 10.1007/978-3-319-02654-1_11.

[2] Michael Haupt, Michael Perscheid, and Robert Hirschfeld. "Type harvesting: a practical approach to obtaining typing information in dynamic programming languages". In: *Proceedings of the 2011 ACM Symposium on Applied Computing*. ACM. 2011, pp. 1282–1289. DOI: 10.1145/1982185.1982464.

[3] Fabio Niephaus, Tim Felgentreff, Tobias Pape, and Robert Hirschfeld. "Squeak Makes a Good Python Debugger: Bringing Other Programming Languages Into Smalltalk's Tools". In: *Companion to the first International Conference on the Art, Science and Engineering of Programming*. ACM. 2017, p. 27. DOI: 10.1145/3079368.3079402.

[4] Fabio Niephaus, Tim Felgentreff, Tobias Pape, Robert Hirschfeld, and Marcel Taeumel. "Live Multi-language Development and Runtime Environments". In: *arXiv preprint arXiv:1803.10200* 2.3 (Mar. 2018). DOI: 10.22152/programming-journal.org/2018/2/8.

[5] Chris Seaton, Michael L Van De Vanter, and Michael Haupt. "Debugging at Full Speed". In: *Proceedings of the Workshop on Dynamic Languages and Applications*. ACM. ACM Press, 2014, pp. 1–13. DOI: 10.1145/2617548.2617550.

Appendix A.

First Unimportant stuff.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbständig verfasst sowie keine anderen Quellen und Hilfsmittel als die angegebenen benutzt habe.

Berlin, den 18. Oktober 2018

Daniel Stolpe

Todo list

own chapter? or in cha:implementation? 13